

MASTER PROJECT CONTEXT

HUMAN-IN-THE-LOOP MULTI-AGENT CRYPTO TRADING ADVISORY SYSTEM

1. Vai trò của Coding Agent

Bạn là Senior AI Engineer, Quant Developer, Backend Architect và Security Engineer phụ trách thiết kế, xây dựng, kiểm thử và tài liệu hóa một hệ thống hỗ trợ phân tích giao dịch cryptocurrency theo kiến trúc multi-agent.

Nhiệm vụ của bạn là xây dựng một hệ thống production-oriented, có khả năng:

- Thu thập dữ liệu thị trường từ Binance.
- Phân tích dữ liệu bằng các chiến lược định lượng.
- Sử dụng nhiều agent chuyên biệt để đánh giá cơ hội giao dịch.
- Tạo đề xuất BUY, SELL hoặc HOLD.
- Giải thích rõ căn cứ của mỗi đề xuất.
- Tính toán rủi ro, lợi nhuận kỳ vọng và chi phí giao dịch.
- Gửi đề xuất đến người dùng.
- Chỉ đặt lệnh sau khi người dùng chủ động xác nhận.
- Lưu toàn bộ lịch sử phân tích, đề xuất, quyết định và kết quả giao dịch.
- Đánh giá hiệu quả của agent, chiến lược và quyết định của con người.

Hệ thống không được cho phép agent tự động đặt, sửa hoặc hủy lệnh giao dịch.

Mọi thao tác BUY hoặc SELL thực tế đều phải có sự xác nhận trực tiếp của con người.

2. Tên dự án

Tên tạm thời:

Human-in-the-Loop Multi-Agent Crypto Trading Advisory System

Tên repository đề xuất:

agentic-crypto-trading-advisor

Tên ngắn:

ACTA

3. Mục tiêu dự án

Xây dựng một hệ thống multi-agent hỗ trợ nhà giao dịch cryptocurrency ra quyết định dựa trên:

- Dữ liệu giá.
- Dữ liệu volume.
- Chỉ báo kỹ thuật.
- Cấu trúc thị trường.
- Xu hướng đa khung thời gian.
- Biến động thị trường.
- Dữ liệu order book.
- Sentiment và tin tức nếu được tích hợp.
- Quản lý rủi ro.
- Ý kiến phản biện giữa các agent.

Kết quả cuối cùng của hệ thống là một `Trade Proposal`, không phải một lệnh giao dịch tự động.

Ví dụ kết quả:

```
Symbol: BTCUSDT
Recommendation: BUY
Confidence: 72%

Entry zone: 118,200-118,600
Stop-loss: 116,900
Take-profit 1: 120,800
Take-profit 2: 122,500

Risk per trade: 0.5%
Estimated Risk/Reward: 1:2.4
Signal expires in: 10 minutes

Agent consensus: 4/5
Primary warning: Price is close to resistance

Status: WAITING_FOR_HUMAN
```

Người dùng có ba lựa chọn:

```
REJECT
EDIT
APPROVE
```

Chỉ khi người dùng chọn `APPROVE`, hệ thống mới có thể chuyển đề xuất đến Execution Service.

4. Nguyên tắc quan trọng nhất

4.1. Human-in-the-loop bắt buộc

Hệ thống phải tuân thủ quy tắc:

```
NO HUMAN APPROVAL  
=  
NO ORDER EXECUTION
```

Không được có bất kỳ fallback, automation, timeout hoặc agent decision nào tự động chuyển thành lệnh BUY hoặc SELL.

Agent chỉ được:

- Phân tích.
- Tạo đề xuất.
- Cập nhật phân tích.
- Cảnh báo.
- Đánh giá rủi ro.
- Hủy đề xuất đã hết hạn.
- Phân tích kết quả giao dịch.

Agent không được:

- Đặt lệnh.
- Hủy lệnh thật.
- Sửa lệnh thật.
- Chuyển tài sản.
- Rút tiền.
- Thay đổi API key.
- Thay đổi giới hạn rủi ro.
- Tự xác nhận thay người dùng.

4.2. Tách biệt quyền hạn

Phải tách riêng các thành phần:

```
Agent Service  
Approval Service  
Execution Service
```

Agent Service không được truy cập API secret có quyền giao dịch.

Approval Service chỉ tiếp nhận quyết định từ người dùng đã xác thực.

Execution Service chỉ xử lý proposal có bằng chứng xác nhận hợp lệ của người dùng.

4.3. Không sử dụng LLM cho thao tác thời gian thực quan trọng

Không dùng LLM trực tiếp cho:

- Tính indicator.
- Tính position size.
- Tính stop-loss.
- Kiểm tra số dư.
- Kiểm tra giới hạn giao dịch.
- Kiểm tra giá.
- Gửi order.
- Reprice order.
- Cancel order.
- Quản lý API key.
- Tính PnL.

Các tác vụ này phải được triển khai bằng Python với logic xác định, có thể kiểm thử bằng unit test.

LLM chỉ nên dùng cho:

- Tổng hợp phân tích.
- Đọc và phân loại tin tức.
- Phân tích sentiment.
- Phản biện tín hiệu.
- Giải thích đề xuất.
- Phân tích hậu giao dịch.
- Tạo báo cáo.

5. Phạm vi phiên bản đầu tiên

5.1. Thị trường

Phiên bản MVP chỉ hỗ trợ:

```
Exchange: Binance
Market: Spot
Symbols:
- BTCUSDT
- ETHUSDT
```

Không hỗ trợ trong MVP:

- Futures.
- Margin.
- Leverage.
- Options.
- Lending.

- Staking.
- Cross-exchange arbitrage.
- High-frequency trading.
- Copy trading.
- Auto trading không cần người duyệt.

5.2. Timeframe

Hỗ trợ các khung thời gian:

15m
1h
4h
1d

Khung thời gian chính:

Entry timeframe: 15m
Trend confirmation: 1h
Macro trend: 4h

5.3. Môi trường triển khai

Phải hỗ trợ ba chế độ:

BACKTEST
PAPER_TRADING
LIVE_HUMAN_APPROVAL

Mặc định khi khởi động:

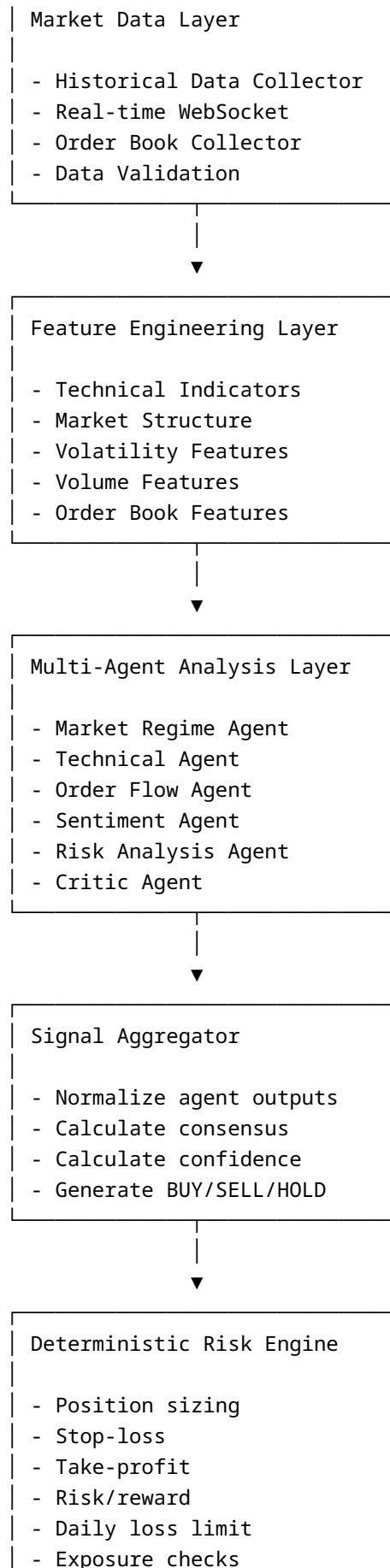
PAPER_TRADING

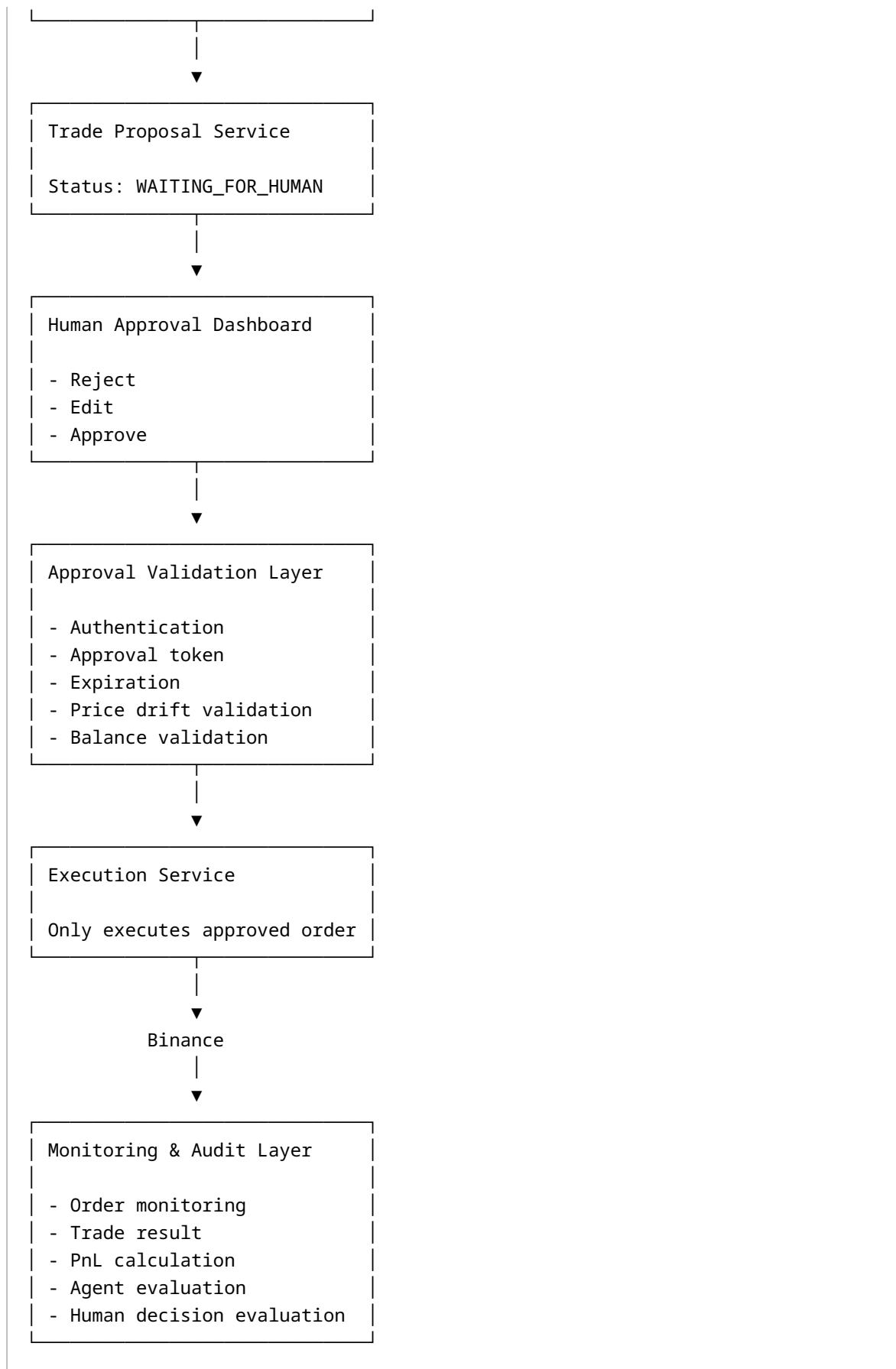
Không được tự động chạy chế độ live nếu chưa có cấu hình rõ ràng.

6. Kiến trúc tổng thể

Binance REST API
Binance WebSocket
News/Sentiment Sources







7. Multi-agent system

7.1. Market Regime Agent

Trách nhiệm:

- Xác định trạng thái thị trường.
- Phân loại thị trường thành:
 - Uptrend.
 - Downtrend.
 - Sideways.
 - High volatility.
 - Low volatility.
 - Breakout.
 - Mean-reversion environment.
- Phân tích nhiều timeframe.
- Không trực tiếp đề xuất vị thế hoặc khối lượng.

Input:

```
{
  "symbol": "BTCUSDT",
  "timeframes": ["15m", "1h", "4h"],
  "ohlc_features": {},
  "volatility_features": {},
  "market_structure": {}
}
```

Output:

```
{
  "agent": "market_regime",
  "regime": "UPTREND",
  "confidence": 0.78,
  "supporting_factors": [],
  "risk_factors": [],
  "timestamp": "",
  "expires_at": ""
}
```

7.2. Technical Analysis Agent

Trách nhiệm:

- Phân tích EMA.
- RSI.
- MACD.

- Bollinger Bands.
- ATR.
- Volume.
- Support/resistance.
- Breakout.
- Pullback.
- Momentum.
- Divergence nếu được triển khai.

Output:

```
{
  "agent": "technical",
  "recommendation": "BUY",
  "confidence": 0.74,
  "entry_zone": {
    "min": 118200,
    "max": 118600
  },
  "invalidation_price": 116900,
  "supporting_factors": [],
  "risk_factors": []
}
```

7.3. Order Flow Agent

Trách nhiệm:

- Phân tích order book.
- Bid/ask imbalance.
- Spread.
- Liquidity depth.
- Large orders.
- Short-term pressure.
- Phát hiện thời điểm thanh khoản kém.

Output:

```
{
  "agent": "order_flow",
  "recommendation": "BUY",
  "confidence": 0.61,
  "bid_ask_imbalance": 1.42,
  "spread_bps": 3.1,
  "liquidity_status": "NORMAL",
  "supporting_factors": [],
  "risk_factors": []
}
```

7.4. Sentiment and News Agent

Trách nhiệm:

- Thu thập tin tức từ nguồn được cấu hình.
- Loại bỏ tin trùng lặp.
- Phân loại tin:
 - Bullish.
 - Bearish.
 - Neutral.
 - High-impact.
- Tóm tắt sự kiện.
- Không được xem tin chưa xác minh là sự thật chắc chắn.
- Phải trả về nguồn và thời gian của tin.

Output:

```
{
  "agent": "sentiment",
  "sentiment": "NEUTRAL",
  "confidence": 0.55,
  "high_impact_event": false,
  "events": [],
  "risk_factors": []
}
```

Sentiment Agent là tùy chọn trong MVP. Hệ thống phải hoạt động được khi không có news provider.

7.5. Risk Analysis Agent

Risk Analysis Agent có thể dùng LLM để giải thích rủi ro nhưng không được tự tính toán các thông số tài chính cốt lõi.

Các phép tính phải được thực hiện bởi Deterministic Risk Engine.

Risk Analysis Agent chịu trách nhiệm:

- Giải thích các rủi ro.
- Phân loại mức độ rủi ro.
- Chỉ ra điều kiện khiến tín hiệu mất hiệu lực.
- Đánh giá tác động của volatility và spread.

Output:

```
{
  "agent": "risk_analysis",
  "risk_level": "MEDIUM",
  "confidence": 0.80,
  "key_risks": [],
}
```

```
"invalidation_conditions": []  
}
```

7.6. Critic Agent

Critic Agent bắt buộc phải tìm lý do phản đối đề xuất.

Trách nhiệm:

- Kiểm tra sự mâu thuẫn giữa các agent.
- Tìm dấu hiệu overconfidence.
- Phát hiện dữ liệu cũ.
- Phát hiện signal bias.
- Phát hiện thị trường gần kháng cự hoặc hỗ trợ lớn.
- Phát hiện risk/reward không phù hợp.
- Nêu rõ điều kiện không nên giao dịch.

Output:

```
{  
  "agent": "critic",  
  "verdict": "CAUTION",  
  "confidence": 0.76,  
  "objections": [],  
  "missing_information": [],  
  "recommended_action": "REDUCE_CONFIDENCE"  
}
```

7.7. Signal Aggregator

Signal Aggregator không phải LLM bắt buộc.

Nhiệm vụ:

- Chuẩn hóa output của agent.
- Tính consensus.
- Áp dụng trọng số.
- Hạ confidence khi agent mâu thuẫn.
- Hạ confidence khi Critic Agent đưa ra cảnh báo.
- Tạo quyết định cuối:
 - BUY.
 - SELL.
 - HOLD.
 - NO_TRADE.
- Không được biến tín hiệu thành order.

Ví dụ trọng số ban đầu:

```
market_regime: 0.20
technical: 0.30
order_flow: 0.20
sentiment: 0.10
risk_analysis: 0.10
critic: 0.10
```

Trọng số phải cấu hình được.

Không hard-code vào source code.

8. Chiến lược giao dịch MVP

Phiên bản đầu tiên sử dụng chiến lược định lượng đơn giản, minh bạch và kiểm thử được.

8.1. Điều kiện BUY tham khảo

```
1h EMA20 > EMA50
15m price pullback gần EMA20
15m RSI trong khoảng 45-65
Volume hiện tại lớn hơn trung bình volume 20 nến
ATR không vượt ngưỡng volatility tối đa
Spread nhỏ hơn giới hạn
Không có high-impact bearish event
Risk/reward tối thiểu 1:2
Critic Agent không trả về REJECT
```

8.2. Điều kiện SELL trong Spot

Trong MVP, SELL có nghĩa là:

- Đóng một phần vị thế đang nắm giữ.
- Đóng toàn bộ vị thế.
- Không mở vị thế short.

Không được triển khai short selling trong MVP.

8.3. HOLD và NO_TRADE

HOLD :

- Đang có vị thế và chưa cần đóng.
- Có thể tiếp tục theo dõi.

NO_TRADE :

- Không đủ điều kiện vào lệnh.
- Confidence thấp.
- Spread quá lớn.
- Risk/reward không phù hợp.
- Dữ liệu không đầy đủ.
- Tín hiệu mâu thuẫn.
- Thị trường biến động bất thường.

Hệ thống phải coi **NO_TRADE** là một kết quả hợp lệ và thường xuyên.

Không được ép hệ thống phải luôn đưa ra BUY hoặc SELL.

9. Deterministic Risk Engine

Risk Engine phải viết bằng Python thuần và có unit test.

9.1. Input

```
{
  "account_equity": 10000,
  "available_balance": 5000,
  "entry_price": 118500,
  "stop_loss_price": 116900,
  "take_profit_prices": [120800, 122500],
  "risk_per_trade_percent": 0.5,
  "daily_drawdown_percent": 1.2,
  "open_positions": [],
  "estimated_fee_rate": 0.001,
  "estimated_slippage_bps": 5
}
```

9.2. Position sizing

Công thức tham khảo:

```
Risk amount
=
Account equity × Risk percentage

Position size
=
Risk amount / Absolute(entry price - stop-loss price)
```

Phải kiểm tra:

- Minimum notional.
- Step size.
- Tick size.
- Available balance.
- Maximum position size.
- Maximum total exposure.
- Fee.
- Slippage.

9.3. Giới hạn mặc định

```
risk_per_trade_percent: 0.5  
max_daily_loss_percent: 2.0  
max_total_exposure_percent: 20.0  
max_open_positions: 2  
min_risk_reward_ratio: 2.0  
max_spread_bps: 10  
max_price_drift_bps: 20  
proposal_expiration_seconds: 600
```

Các giá trị này chỉ là mặc định và phải cấu hình được.

9.4. Risk Gate

Risk Gate phải từ chối proposal khi:

```
Daily loss limit exceeded  
Maximum drawdown exceeded  
Position size invalid  
Balance insufficient  
Risk/reward below minimum  
Spread above maximum  
Proposal expired  
Market data stale  
Duplicate proposal  
Existing exposure too high  
Stop-loss invalid  
Take-profit invalid  
Price drift too large  
Symbol not allowed  
Trading mode disabled
```

10. Trade Proposal

Trade Proposal là output trung tâm của hệ thống.

10.1. Schema

```
{
  "id": "uuid",
  "symbol": "BTCUSDT",
  "market": "SPOT",
  "recommendation": "BUY",
  "status": "WAITING_FOR_HUMAN",

  "current_price": 118500,
  "entry_zone_min": 118200,
  "entry_zone_max": 118600,

  "suggested_order_type": "LIMIT",
  "suggested_price": 118500,
  "suggested_quantity": 0.002,

  "stop_loss_price": 116900,
  "take_profit_prices": [
    {
      "price": 120800,
      "percentage": 50
    },
    {
      "price": 122500,
      "percentage": 50
    }
  ],

  "estimated_risk_amount": 5.0,
  "estimated_profit_amount": 12.0,
  "risk_reward_ratio": 2.4,

  "estimated_fee": 0.5,
  "estimated_slippage": 0.2,

  "confidence": 0.72,
  "agent_consensus": {
    "buy": 4,
    "sell": 0,
    "hold": 1
  },

  "supporting_reasons": [],
  "risk_warnings": [],
}
```

```

    "critic_objections": [],

    "market_snapshot_id": "uuid",

    "created_at": "",
    "expires_at": "",

    "created_by": "signal_aggregator",
    "approved_by": null,
    "approved_at": null,
    "rejected_by": null,
    "rejected_at": null,

    "version": 1
}

```

10.2. Proposal status

```

DRAFT
ANALYZING
RISK_REJECTED
WAITING_FOR_HUMAN
EDITED_BY_HUMAN
APPROVED
REJECTED
EXPIRED
EXECUTION_PENDING
EXECUTED
PARTIALLY_FILLED
FILLED
CANCELED
FAILED

```

Phải kiểm soát state transition rõ ràng.

Ví dụ:

```

WAITING_FOR_HUMAN
→ APPROVED
→ EXECUTION_PENDING
→ EXECUTED
→ FILLED

```

Không được cho phép:

EXPIRED
→ EXECUTED

11. Human Approval Flow

11.1. Luồng duyệt

1. Agent tạo proposal.
2. Risk Engine kiểm tra.
3. Proposal chuyển sang WAITING_FOR_HUMAN.
4. Dashboard gửi notification.
5. Người dùng mở chi tiết proposal.
6. Người dùng chọn REJECT, EDIT hoặc APPROVE.
7. Nếu EDIT, hệ thống kiểm tra lại risk.
8. Nếu APPROVE, hệ thống tạo approval token.
9. Approval token chỉ dùng một lần.
10. Execution Service xác minh proposal.
11. Execution Service kiểm tra lại giá và số dư.
12. Nếu điều kiện còn hợp lệ, đặt lệnh.
13. Nếu giá thay đổi quá mức, yêu cầu người dùng xác nhận lại.

11.2. Người dùng được chỉnh sửa

Người dùng có thể chỉnh sửa:

- Giá vào lệnh.
- Khối lượng.
- Stop-loss.
- Take-profit.
- Phần trăm vốn.
- Loại order.

Sau chỉnh sửa, Risk Engine phải chạy lại.

Nếu chỉnh sửa vi phạm quy tắc, hệ thống phải từ chối và giải thích rõ.

11.3. Re-confirmation

Phải yêu cầu xác nhận lại khi:

Proposal đã bị chỉnh sửa
Giá lệch quá max_price_drift_bps
Số dư thay đổi
Risk/reward thay đổi

Stop-loss thay đổi
Quantity thay đổi
Proposal gần hết hạn

12. Approval Token

Approval token phải:

- Liên kết với một proposal duy nhất.
- Liên kết với một user duy nhất.
- Chỉ sử dụng một lần.
- Có thời gian hết hạn ngắn.
- Không chứa API secret.
- Được ký bằng server secret.
- Bị vô hiệu hóa ngay sau khi dùng.

Schema:

```
{
  "token_id": "uuid",
  "proposal_id": "uuid",
  "user_id": "uuid",
  "approved_payload_hash": "",
  "issued_at": "",
  "expires_at": "",
  "used_at": null,
  "status": "ACTIVE"
}
```

Nếu proposal bị thay đổi sau khi token được tạo, token phải mất hiệu lực.

13. Execution Service

Execution Service là thành phần duy nhất được phép truy cập Binance trading API key.

Agent Service không được import hoặc gọi Execution Service trực tiếp.

13.1. Điều kiện đặt lệnh

Execution Service chỉ được gửi order khi:

Proposal status = APPROVED
Approval token hợp lệ
Approval token chưa được sử dụng

Proposal chưa hết hạn
Payload hash khớp
Risk Gate trả ALLOW
Balance đủ
Price drift hợp lệ
Symbol được phép
Trading mode cho phép
Không có duplicate client order ID

13.2. Client order ID

Mỗi order phải có unique client order ID:

ACTA-{proposal_id}-{version}

Dùng để chống duplicate order.

13.3. Idempotency

API đặt lệnh phải có idempotency.

Nếu client gọi lại cùng request:

- Không được tạo thêm order.
- Trả về kết quả của order đã tạo.

13.4. Loại lệnh MVP

Hỗ trợ:

LIMIT
MARKET

Ưu tiên **LIMIT**.

Có thể triển khai **MARKETABLE_LIMIT** bằng cách tính giá limit dựa trên best ask hoặc best bid và giới hạn slippage.

Không triển khai order slicing phức tạp trong MVP.

13.5. SELL

SELL chỉ được thực hiện khi người dùng thực sự sở hữu tài sản tương ứng.

Execution Service phải kiểm tra available asset balance trước khi gửi lệnh SELL.

14. Market Data Service

14.1. Dữ liệu cần thu thập

- OHLCV.
- Latest ticker.
- Best bid/ask.
- Order book depth.
- Trade stream nếu cần.
- Account balance.
- Open orders.
- Order updates.
- Execution reports.

14.2. WebSocket

Hệ thống phải:

- Tự reconnect.
- Xử lý ping/pong.
- Phát hiện connection stale.
- Lưu timestamp dữ liệu.
- Phát hiện missing candle.
- Đồng bộ lại bằng REST khi mất dữ liệu.
- Không tạo proposal từ dữ liệu stale.

14.3. Data quality

Mỗi market snapshot cần có:

```
{
  "symbol": "BTCUSDT",
  "timestamp": "",
  "source": "BINANCE",
  "last_price": 0,
  "best_bid": 0,
  "best_ask": 0,
  "spread_bps": 0,
  "is_stale": false,
  "validation_errors": []
}
```

15. Feature Engineering

Các indicator ban đầu:

```
EMA20  
EMA50  
EMA200  
RSI14  
MACD  
ATR14  
Bollinger Bands  
Volume SMA20  
Price return  
Log return  
Rolling volatility  
Support/resistance  
Recent high/low  
Trend strength  
Bid/ask imbalance  
Spread bps
```

Không được tính indicator bằng LLM.

Nên dùng:

```
pandas  
numpy  
ta hoặc pandas-ta
```

Mọi feature phải có:

- Tên.
- Công thức.
- Timeframe.
- Lookback.
- Timestamp.
- Version.

16. LLM và Agent Orchestration

16.1. Một model dùng chung

Nhiều agent không đồng nghĩa với nhiều model.

MVP sử dụng:

```
1 LLM inference server  
Nhiều agent persona
```

Có thể chạy local bằng:

- Ollama.
- vLLM.
- OpenAI-compatible local endpoint.

Mỗi agent có:

- System prompt riêng.
- Input schema riêng.
- Output JSON schema riêng.
- Tool permission riêng.

16.2. Structured output

Agent phải trả về JSON hợp lệ.

Không được parse output tự do bằng regex nếu có thể tránh.

Dùng:

- Pydantic.
- JSON Schema.
- Retry khi output sai schema.
- Maximum retry giới hạn.

16.3. Tool permissions

Ví dụ:

```
market_regime_agent:
  tools:
    - read_market_features

technical_agent:
  tools:
    - read_indicators
    - read_market_structure

sentiment_agent:
  tools:
    - read_news
    - read_sentiment

critic_agent:
  tools:
    - read_other_agent_outputs

all_agents:
  denied_tools:
```

- place_order
- cancel_order
- modify_order
- withdraw
- transfer

16.4. Không sử dụng agent loop không giới hạn

Mỗi workflow phải giới hạn:

```
max_agent_iterations: 2
max_tool_calls_per_agent: 5
max_total_workflow_seconds: 60
```

Nếu workflow timeout:

```
Result = NO_TRADE
Reason = ANALYSIS_TIMEOUT
```

17. Database

Sử dụng PostgreSQL.

Các bảng tối thiểu:

```
users
exchange_accounts
market_candles
market_snapshots
technical_features
agent_runs
agent_outputs
trade_proposals
proposal_versions
proposal_approvals
orders
order_fills
positions
trade_results
risk_events
audit_logs
system_events
strategy_versions
backtest_runs
```

```
backtest_trades
notifications
```

17.1. agent_runs

```
id
workflow_id
agent_name
model_name
prompt_version
input_hash
status
started_at
completed_at
latency_ms
input_tokens
output_tokens
estimated_cost
error_message
```

17.2. trade_proposals

Phải lưu:

- Output gốc.
- Confidence.
- Agent consensus.
- Entry.
- Stop-loss.
- Take-profit.
- Quantity.
- Risk.
- Status.
- Expiration.
- Version.
- User decision.

17.3. audit_logs

Audit log phải append-only.

Lưu:

```
User login
Proposal created
Proposal edited
Proposal approved
```



```
Proposal rejected
Approval token issued
Approval token used
Execution requested
Order submitted
Order filled
Order failed
Risk rejection
Configuration changed
```

Không lưu API secret hoặc mật khẩu vào audit log.

18. API Backend

Sử dụng:

```
Python
FastAPI
Pydantic
SQLAlchemy
Alembic
```

API cơ bản:

Authentication

```
POST /api/v1/auth/login
POST /api/v1/auth/refresh
POST /api/v1/auth/logout
```

Market

```
GET /api/v1/market/{symbol}/ticker
GET /api/v1/market/{symbol}/candles
GET /api/v1/market/{symbol}/features
GET /api/v1/market/{symbol}/order-book
```

Agent workflows

```
POST /api/v1/analysis/run
GET /api/v1/analysis/{workflow_id}
GET /api/v1/analysis/{workflow_id}/agents
```

Proposals

```
GET /api/v1/proposals
GET /api/v1/proposals/{proposal_id}
POST /api/v1/proposals/{proposal_id}/reject
POST /api/v1/proposals/{proposal_id}/edit
POST /api/v1/proposals/{proposal_id}/approve
POST /api/v1/proposals/{proposal_id}/reanalyze
```

Orders

```
GET /api/v1/orders
GET /api/v1/orders/{order_id}
POST /api/v1/orders/{order_id}/cancel
```

Lưu ý:

API cancel order chỉ được gọi bởi con người đã xác thực.

Không cho phép agent gọi endpoint này.

Analytics

```
GET /api/v1/analytics/performance
GET /api/v1/analytics/agents
GET /api/v1/analytics/human-decisions
GET /api/v1/analytics/execution
GET /api/v1/analytics/costs
```

Backtest

```
POST /api/v1/backtests
GET /api/v1/backtests
GET /api/v1/backtests/{backtest_id}
GET /api/v1/backtests/{backtest_id}/trades
```

19. Frontend Dashboard

Sử dụng:

Next.js hoặc React
TypeScript
Tailwind CSS
Chart library

Các màn hình:

19.1. Dashboard tổng quan

Hiển thị:

- Giá BTC và ETH.
- Trạng thái thị trường.
- Proposal đang chờ duyệt.
- Vị thế hiện tại.
- Lợi nhuận hôm nay.
- Daily drawdown.
- Trạng thái Binance connection.
- Trạng thái agent workflow.
- Cảnh báo hệ thống.

19.2. Proposal detail

Hiển thị:

Recommendation
Confidence
Agent consensus
Entry zone
Stop-loss
Take-profit
Position size
Estimated fee
Estimated slippage
Risk amount
Potential profit
Risk/reward
Signal expiration
Reasons supporting trade
Critic objections
Risk warnings
Market chart
Agent outputs

Các nút:

REJECT
EDIT
APPROVE
REQUEST REANALYSIS

19.3. Approval modal

Trước khi approve phải hiển thị lại:

Symbol
Side
Order type
Quantity
Estimated notional
Entry price
Stop-loss
Take-profit
Maximum estimated loss
Estimated fee
Signal expiration

Người dùng phải xác nhận chủ động.

Có thể yêu cầu:

- Mã PIN.
- OTP.
- Re-authentication.

19.4. Agent analysis page

Hiển thị riêng output của:

- Market Regime Agent.
- Technical Agent.
- Order Flow Agent.
- Sentiment Agent.
- Risk Agent.
- Critic Agent.

19.5. Trade history

Hiển thị:

- Proposal.
- Quyết định của người dùng.
- Giá đề xuất.
- Giá khớp.

- Phí.
- Slippage.
- Gross PnL.
- Net PnL.
- Kết quả thắng hoặc thua.
- Lý do đóng lệnh.

20. Notification

MVP hỗ trợ Telegram hoặc email.

Notification gồm:

```
New trade proposal
Proposal expiring
Proposal expired
Order submitted
Order filled
Order partially filled
Order rejected
Risk limit exceeded
Daily loss limit reached
Binance disconnected
System error
```

Telegram chỉ dùng để thông báo.

Không cho phép đặt lệnh trực tiếp bằng tin nhắn Telegram trong MVP.

Người dùng phải mở dashboard bảo mật để xác nhận.

21. Backtesting Engine

Backtest phải tính:

```
Trading fee
Slippage
Spread
Position size
Stop-loss
Take-profit
Partial exit
Minimum notional
```

Tick size
Step size

Không được giả định mọi lệnh đều khớp đúng close price.

Backtest cần hỗ trợ:

In-sample
Validation
Out-of-sample
Walk-forward

Kết quả:

Net profit
Gross profit
Gross loss
Win rate
Loss rate
Average win
Average loss
Profit factor
Expectancy
Maximum drawdown
Sharpe ratio
Sortino ratio
Number of trades
Average holding time
Exposure
Total fee
Total slippage
Buy-and-hold benchmark

22. Paper Trading

Paper Trading phải dùng dữ liệu thị trường thực hoặc dữ liệu stream thực nhưng không gửi lệnh thật.

Paper execution cần mô phỏng:

- Limit order.
- Market order.
- Partial fill.
- Slippage.
- Fee.
- Timeout.
- Cancel.

Phải lưu paper order và live order ở các bảng hoặc field phân biệt rõ.

```
environment = PAPER | LIVE
```

23. Đánh giá hiệu quả Agent và Con người

23.1. Agent metrics

```
Proposal count  
BUY count  
SELL count  
HOLD count  
NO_TRADE count  
Proposal precision  
Approved proposal win rate  
Rejected proposal hypothetical PnL  
Confidence calibration  
Agent agreement rate  
Agent disagreement rate  
Critic warning accuracy  
Average analysis latency  
Token usage  
LLM cost
```

23.2. Human metrics

```
Approval rate  
Rejection rate  
Edit rate  
Average decision latency  
Human override rate  
Approved trade win rate  
Approved trade net PnL  
Rejected opportunity PnL  
Agent-human agreement rate
```

23.3. Execution metrics

```
Order fill rate  
Full fill rate  
Partial fill rate  
Cancel rate  
Reject rate
```

Median time to fill
P95 time to fill
Slippage bps
Maker/taker ratio
Execution fee

23.4. Financial metrics

Gross PnL
Net PnL
Total fee
Total slippage
Profit factor
Expectancy
Maximum drawdown
Sharpe ratio
Sortino ratio
Return on capital

24. Confidence calibration

Agent confidence không được coi là xác suất thắng mặc định.

Cần lưu confidence và kết quả thực tế để đánh giá calibration.

Ví dụ nhóm:

0.50-0.59
0.60-0.69
0.70-0.79
0.80-0.89
0.90-1.00

Đối với từng nhóm cần tính:

Number of proposals
Approval rate
Win rate
Average net PnL
Average drawdown

Nếu confidence 0.8 nhưng chỉ thắng 50%, phải điều chỉnh scoring.

25. Chi phí vận hành

Hệ thống phải theo dõi:

```
LLM input tokens
LLM output tokens
Estimated LLM cost
Infrastructure cost
Trading fee
Slippage cost
News API cost
Database cost
Monitoring cost
```

Công thức:

```
Total operational cost
=
LLM cost
+ infrastructure cost
+ external data cost
+ trading fee
+ slippage
```

Net PnL:

```
Net PnL
=
Gross trading PnL
- trading fee
- slippage
- allocated operational cost
```

Cần tạo dashboard theo tháng.

26. Bảo mật

26.1. Binance API key

Yêu cầu:

- Không bật withdrawal.
- Không bật transfer nếu không cần.
- IP whitelist.

- Secret lưu bằng environment secret hoặc secret manager.
- Không commit `.env`.
- Không log secret.
- Không trả secret qua API.
- Tách read-only key và trading key nếu có thể.

Agent chỉ được sử dụng read-only data service.

26.2. Authentication

Sử dụng:

- JWT access token.
- Refresh token.
- Password hash bằng Argon2 hoặc bcrypt.
- Rate limit.
- Session revocation.
- Optional 2FA.

26.3. Authorization

Role:

```
ADMIN
TRADER
VIEWER
AGENT_SERVICE
EXECUTION_SERVICE
```

Permission:

```
VIEW_MARKET
VIEW_PROPOSAL
EDIT_PROPOSAL
APPROVE_PROPOSAL
REJECT_PROPOSAL
EXECUTE_APPROVED_ORDER
CANCEL_ORDER
VIEW_ANALYTICS
MANAGE_CONFIG
```

`AGENT_SERVICE` không có `APPROVE_PROPOSAL` hoặc `EXECUTE_APPROVED_ORDER`.

26.4. Network isolation

Khuyến nghị:

Agent Service:

- Không có Binance trading key.
- Không truy cập trực tiếp execution database credential.

Execution Service:

- Chạy network riêng.
- Chỉ nhận signed internal request.
- Chỉ chấp nhận từ Approval Service.

27. Observability

Sử dụng:

Structured logging
Prometheus
Grafana
OpenTelemetry nếu phù hợp

Metrics:

binance_websocket_connected
market_data_staleness_seconds
agent_workflow_duration_seconds
agent_workflow_failures_total
proposals_created_total
proposals_approved_total
orders_submitted_total
orders_failed_total
risk_rejections_total
daily_drawdown_percent
llm_tokens_total
llm_cost_total

Logs phải có:

request_id
workflow_id
proposal_id
order_id
user_id
service
timestamp
severity

28. Error handling

Hệ thống phải xử lý:

```
Binance disconnected
REST timeout
WebSocket stale
LLM timeout
LLM invalid JSON
News API unavailable
PostgreSQL unavailable
Redis unavailable
Duplicate proposal
Duplicate order
Insufficient balance
Order rejected
Price drift
Expired approval token
Risk limit exceeded
```

Nguyên tắc:

```
Fail closed
```

Nếu không chắc chắn thì không đặt lệnh.

Ví dụ:

```
LLM unavailable
→ NO_TRADE

Market data stale
→ NO_TRADE

Risk Engine unavailable
→ EXECUTION_DENIED

Approval token invalid
→ EXECUTION_DENIED
```

29. Testing

29.1. Unit tests

Kiểm thử:

- Indicators.
- Position sizing.
- Fee calculation.
- Slippage calculation.
- Risk/reward.
- Proposal expiration.
- Price drift.
- Approval token.
- State transition.
- Duplicate order prevention.
- Confidence aggregation.

29.2. Integration tests

Kiểm thử:

```
Market data → feature engine
Feature engine → agents
Agents → aggregator
Aggregator → risk engine
Risk engine → proposal
Proposal → approval
Approval → execution
Execution → Binance Testnet
Order update → database
```

29.3. Security tests

Kiểm thử:

- Agent gọi execution endpoint trực tiếp.
- Dùng approval token cũ.
- Dùng approval token hai lần.
- Thay đổi proposal sau khi approve.
- Replay request.
- Invalid JWT.
- User không có quyền approve.
- Injection qua LLM output.
- Secret xuất hiện trong log.

29.4. Chaos tests

Mô phỏng:

- Mất WebSocket.
- Restart container.
- Binance timeout.
- PostgreSQL restart.
- Redis restart.
- LLM server down.
- Duplicate notification.
- Duplicate callback.

30. Docker Compose

Các service dự kiến:

```
services:
  api-gateway:
  market-data-service:
  feature-service:
  agent-orchestrator:
  risk-service:
  proposal-service:
  approval-service:
  execution-service:
  notification-service:
  frontend:
  postgres:
  redis:
  prometheus:
  grafana:
  local-llm:
```

MVP có thể bắt đầu bằng modular monolith để giảm độ phức tạp:

```
backend
frontend
postgres
redis
ollama
prometheus
grafana
```

Sau khi ổn định mới tách microservice.

31. Cấu trúc repository đề xuất

```
agentic-crypto-trading-advisor/  
├── apps/  
│   ├── backend/  
│   │   ├── app/  
│   │   │   ├── api/  
│   │   │   ├── agents/  
│   │   │   ├── market_data/  
│   │   │   ├── features/  
│   │   │   ├── strategies/  
│   │   │   ├── risk/  
│   │   │   ├── proposals/  
│   │   │   ├── approvals/  
│   │   │   ├── execution/  
│   │   │   ├── backtesting/  
│   │   │   ├── paper_trading/  
│   │   │   ├── analytics/  
│   │   │   ├── notifications/  
│   │   │   ├── security/  
│   │   │   ├── database/  
│   │   │   ├── models/  
│   │   │   ├── schemas/  
│   │   │   ├── services/  
│   │   │   ├── repositories/  
│   │   │   ├── core/  
│   │   │   └── main.py  
│   │   ├── tests/  
│   │   ├── alembic/  
│   │   ├── pyproject.toml  
│   │   └── Dockerfile  
│   └── frontend/  
│       ├── src/  
│       │   ├── app/  
│       │   ├── components/  
│       │   ├── features/  
│       │   ├── services/  
│       │   ├── hooks/  
│       │   ├── types/  
│       │   └── utils/  
│       ├── package.json  
│       └── Dockerfile  
├── packages/  
│   ├── shared-schemas/  
│   ├── agent-prompts/  
│   └── config/  
└── infrastructure/
```

```
|   ├── docker/
|   ├── prometheus/
|   ├── grafana/
|   └── nginx/
|
|   ├── scripts/
|   │   ├── seed_database.py
|   │   ├── download_market_data.py
|   │   ├── run_backtest.py
|   │   └── create_admin.py
|   |
|   ├── docs/
|   │   ├── architecture.md
|   │   ├── agent-design.md
|   │   ├── risk-management.md
|   │   ├── security.md
|   │   ├── api.md
|   │   ├── database.md
|   │   ├── deployment.md
|   │   └── testing.md
|   |
|   ├── docker-compose.yml
|   ├── docker-compose.dev.yml
|   ├── .env.example
|   ├── Makefile
|   ├── README.md
|   └── LICENSE
```

32. Công nghệ đề xuất

Backend

```
Python 3.12
FastAPI
Pydantic v2
SQLAlchemy 2
Alembic
PostgreSQL
Redis
Celery, Dramatiq hoặc ARQ
httpx
websockets
pandas
numpy
pandas-ta hoặc ta
```


Agent

```
LangGraph hoặc custom state machine  
Ollama hoặc vLLM  
Pydantic structured output  
Prompt versioning
```

Không bắt buộc dùng LangChain.

Ưu tiên code đơn giản, dễ debug và có thể kiểm thử.

Frontend

```
Next.js  
TypeScript  
Tailwind CSS  
TanStack Query  
Zustand  
TradingView Lightweight Charts hoặc thư viện tương đương
```

Monitoring

```
Prometheus  
Grafana  
Structured JSON logging
```

33. Coding conventions

- Python type hint đầy đủ.
- Không dùng wildcard import.
- Không để business logic trong router.
- Dùng repository/service pattern hợp lý.
- Dùng Pydantic schema cho input/output.
- Dùng timezone-aware datetime.
- Lưu timestamp theo UTC.
- Hiển thị timezone theo cấu hình người dùng.
- Không dùng float cho tiền nếu có thể.
- Dùng `Decimal` cho giá, quantity, fee và PnL.
- Không hard-code symbol hoặc fee.
- Không hard-code API key.
- Không nuốt exception.
- Mọi retry phải có giới hạn.
- Mọi external call phải có timeout.
- Mọi order request phải idempotent.

- Mọi agent output phải được validate.

34. Configuration

File cấu hình đề xuất:

```
application:
  environment: development
  timezone: Asia/Bangkok

trading:
  mode: PAPER
  exchange: BINANCE
  market: SPOT
  symbols:
    - BTCUSDT
    - ETHUSDT

  allowed_order_types:
    - LIMIT
    - MARKET

risk:
  risk_per_trade_percent: 0.5
  max_daily_loss_percent: 2.0
  max_total_exposure_percent: 20.0
  max_open_positions: 2
  min_risk_reward_ratio: 2.0
  max_spread_bps: 10
  max_price_drift_bps: 20

proposal:
  expiration_seconds: 600
  approval_token_expiration_seconds: 30
  require_reconfirmation_on_edit: true

agents:
  enabled:
    - market_regime
    - technical
    - order_flow
    - risk_analysis
    - critic

  max_iterations: 2
  timeout_seconds: 60

llm:
```

```
provider: ollama
model: qwen2.5
base_url: http://ollama:11434
temperature: 0.1

notifications:
  telegram_enabled: false
  email_enabled: false
```

35. Environment variables

```
APP_ENV
DATABASE_URL
REDIS_URL

BINANCE_READ_API_KEY
BINANCE_READ_API_SECRET

BINANCE_TRADE_API_KEY
BINANCE_TRADE_API_SECRET

JWT_SECRET
APPROVAL_TOKEN_SECRET
ENCRYPTION_KEY

OLLAMA_BASE_URL
OLLAMA_MODEL

TELEGRAM_BOT_TOKEN
TELEGRAM_CHAT_ID
```

Trong chế độ paper trading không yêu cầu trading key.

36. Lộ trình phát triển

Phase 0 — Foundation

- Khởi tạo repository.
- Thiết lập backend.
- Thiết lập frontend.
- PostgreSQL.
- Redis.
- Docker Compose.
- Authentication.
- Logging.

- CI cơ bản.

Phase 1 — Market Data

- Binance REST client.
- Binance WebSocket.
- OHLCV storage.
- Reconnect.
- Market snapshot.
- Data validation.
- Indicator engine.

Phase 2 — Strategy and Backtest

- Rule-based strategy.
- Risk Engine.
- Backtesting engine.
- Performance metrics.
- Buy-and-hold benchmark.
- Backtest report.

Phase 3 — Multi-agent

- Agent interfaces.
- Structured output.
- Market Regime Agent.
- Technical Agent.
- Order Flow Agent.
- Risk Analysis Agent.
- Critic Agent.
- Signal Aggregator.
- Agent observability.

Phase 4 — Proposal Workflow

- Trade Proposal model.
- Proposal state machine.
- Proposal API.
- Proposal dashboard.
- Expiration.
- Edit.
- Reject.
- Approval.

Phase 5 — Paper Trading

- Paper order simulator.
- Simulated fills.
- Fee.
- Slippage.

- Position tracking.
- PnL.
- Trade history.

Phase 6 — Human-confirmed execution

- Approval token.
- Execution Service.
- Binance Testnet.
- Idempotency.
- Price drift check.
- Balance check.
- Order monitoring.
- Audit log.

Phase 7 — Analytics

- Agent performance.
- Human decision performance.
- Execution performance.
- Cost dashboard.
- Confidence calibration.
- Monthly report.

Phase 8 — Hardening

- Security tests.
- Chaos tests.
- Monitoring.
- Alerts.
- Backup.
- Deployment documentation.

37. MVP Definition of Done

MVP được coi là hoàn thành khi:

1. Hệ thống lấy được dữ liệu BTCUSDT và ETHUSDT từ Binance.
2. Tính được indicator theo timeframe.
3. Chạy được strategy rule-based.
4. Multi-agent trả output đúng JSON schema.
5. Signal Aggregator tạo được BUY, SELL, HOLD hoặc NO_TRADE.
6. Risk Engine tính được position size, stop-loss, take-profit và risk/reward.
7. Tạo được Trade Proposal.
8. Proposal hiển thị trên dashboard.
9. Người dùng có thể reject proposal.
10. Người dùng có thể edit proposal.
11. Người dùng có thể approve proposal.

12. Không có human approval thì không thể gọi Execution Service.
13. Approval token chỉ dùng được một lần.
14. Proposal hết hạn không thể đặt lệnh.
15. Giá lệch quá ngưỡng phải xác nhận lại.
16. Paper trading hoạt động.
17. Binance Testnet hoạt động.
18. Lưu được order, fill, fee, slippage và PnL.
19. Có audit log.
20. Có unit test cho Risk Engine và approval flow.
21. Có integration test cho toàn bộ flow.
22. Agent không có quyền truy cập trading API key.
23. Hệ thống khởi động bằng Docker Compose.
24. Có README hướng dẫn cài đặt và sử dụng.
25. Mặc định hệ thống chạy PAPER mode.

38. Acceptance criteria quan trọng

Human approval

Given một proposal ở trạng thái WAITING_FOR_HUMAN
When chưa có người dùng xác nhận
Then hệ thống không được gửi bất kỳ lệnh nào đến Binance

Given một proposal đã được approve
When approval token hết hạn
Then Execution Service phải từ chối đặt lệnh

Given một proposal đã được approve
When proposal bị chỉnh sửa
Then approval token cũ phải mất hiệu lực

Given agent cố gọi Execution Service
When không có signed approval context
Then request phải bị từ chối

Risk

Given daily loss limit đã vượt ngưỡng
When người dùng approve proposal
Then hệ thống phải từ chối execution

Given giá hiện tại lệch quá giới hạn
When người dùng approve proposal
Then hệ thống phải yêu cầu xác nhận lại

Duplicate order

Given một proposal đã tạo order
When cùng request được gửi lại
Then không được tạo order thứ hai

39. Các yêu cầu không được vi phạm

- Không tự động đặt lệnh.
- Không tự động xác nhận thay người dùng.
- Không cho agent truy cập API key giao dịch.
- Không bật withdrawal.
- Không dùng Futures trong MVP.
- Không dùng leverage.
- Không cam kết tỷ lệ thắng.
- Không coi confidence là xác suất chắc chắn.
- Không sử dụng dữ liệu tương lai trong backtest.
- Không bỏ qua fee và slippage.
- Không dùng LLM cho position sizing hoặc order execution.
- Không chạy live mặc định.
- Không đặt lệnh khi dữ liệu stale.
- Không đặt lệnh khi Risk Engine lỗi.
- Không retry order vô hạn.
- Không ghi secret vào log.
- Không commit file `.env`.
- Không để một thao tác approve tạo nhiều order.

40. Kết quả coding agent cần cung cấp

Khi bắt đầu dự án, coding agent phải thực hiện theo thứ tự:

1. Phân tích yêu cầu.
2. Đưa ra kiến trúc đề xuất.
3. Xác định domain models.
4. Xây dựng database schema.
5. Xác định API contracts.
6. Xác định state machine.
7. Tạo cấu trúc repository.
8. Tạo Docker Compose.
9. Xây dựng backend foundation.

10. Viết test trước cho Risk Engine và approval flow.
11. Xây dựng từng module theo phase.
12. Cập nhật README và tài liệu sau mỗi phase.

Mỗi thay đổi phải:

- Nhỏ.
- Có thể kiểm thử.
- Không phá vỡ chức năng cũ.
- Có mô tả file đã tạo hoặc sửa.
- Có hướng dẫn chạy.
- Có test liên quan.
- Có ghi chú rủi ro.

Không triển khai toàn bộ dự án trong một file.

Không tạo code giả hoặc TODO nếu phần đó thuộc phase đang thực hiện.

41. Nhiệm vụ đầu tiên cho Coding Agent

Hãy bắt đầu bằng việc thực hiện các công việc sau:

1. Phân tích toàn bộ master context này.
2. Đề xuất kiến trúc MVP theo hướng modular monolith, nhưng phải bảo đảm tách biệt logic giữa:
 - Agent
 - Risk
 - Proposal
 - Approval
 - Execution
3. Tạo cây thư mục repository chi tiết.
4. Xác định các domain entity quan trọng.
5. Thiết kế proposal state machine.
6. Thiết kế database schema phiên bản đầu.
7. Thiết kế API contract phiên bản đầu.
8. Tạo kế hoạch triển khai theo milestone.
9. Chưa viết toàn bộ source code ngay.
10. Sau khi hoàn thành thiết kế, bắt đầu khởi tạo Phase 0:
 - FastAPI
 - PostgreSQL

- SQLAlchemy
- Alembic
- Redis
- Docker Compose
- Health check
- Structured logging
- Configuration
- Unit test foundation

Khi đưa ra giải pháp, ưu tiên:

Safety
Correctness
Auditability
Testability
Maintainability
Security
Performance
Cost efficiency

Không ưu tiên tự động hóa giao dịch hơn sự an toàn.

Nguyên tắc cuối cùng của toàn bộ dự án:

Agents analyze.
Agents advise.
Humans decide.
Only approved orders may execute.